МИНОБРНАУКИ РОССИИ
Санкт-Петербургский государственный
электротехнический университет
«ЛЭТИ» им. В.И. Ульянова (Ленина)
Кафедра информационных систем

отчет
по лабораторной работе №1
по дисциплине «Информатика и информационные технологии»

Тема: Простой интерпретатор арифметических действий
Вариант 10

Студент гр. 6374 _____ Дехтярёв В.Н.

Преподаватель _____ _____

Санкт-Петербург
2026

Цель работы.

Научиться писать простой интерпретатор: написать на языке Python программу, которая принимает строку с арифметическим выражением и вычисляет результат. Программа должна поддерживать операции +, -, *, /, скобки, унарный минус, целые и дробные числа, а по варианту 10 ещё функцию `round(x, n)` и оператор `>=`. Использовать `eval()` и `sympy` нельзя.

Основные теоретические положения.

Интерпретатор — это программа, которая читает инструкции и сразу их выполняет. Простой интерпретатор арифметических выражений работает в два этапа:

строка → **лексер** → токены → **парсер** → число

- Л**ексер** разбивает строку на токены: числа, знаки операций, скобки, имена функций. Пробелы он выбрасывает. Например, «2 + 3 * (4 - 1)» превращается в набор токенов: число 2, плюс, число 3, умножить, открывающая скобка, число 4, минус, число 1, закрывающая скобка.
- **Парсер** проверяет, что токены записаны по правилам, и сразу вычисляет результат. Использован метод рекурсивного спуска: для каждого уровня приоритета написана отдельная функция, а функция с низким приоритетом просит функцию с более высоким посчитать операнды. Поэтому умножение считается раньше сложения.

Приоритет операций от низкого к высокому: сначала **>=**, затем **+ и -**, затем *** и /**, и выше всех скобки, числа и вызовы функций. Операции одного приоритета выполняются слева направо: 10 - 3 - 2 = (10 - 3) - 2 = 5.

Унарный минус и плюс. Знак перед числом считается унарным, если он стоит в начале выражения или сразу после открывающей скобки (например, «-5 + 3» или «(-5) * 2»). По заданию два знака подряд запрещены, поэтому «2 ++ 3», «2 + -3», «2 * -3» и «--5» дают ошибку.

Функция round(x, n) округляет число x до n знаков после запятой, где n — целое число, не меньше нуля. Использована встроенная функция Python round. Если отбрасываемая часть ровно «пять», она округляет к ближайшему чётному числу (round(2.5, 0) даёт 2, а round(3.5, 0) даёт 4). **Оператор >=** («больше или равно») возвращает 1, если условие верно, и 0, если нет, чтобы результат оставался числом.

Экспериментальные результаты.

Программа записана в файле interpreter.py, полный код приведён в приложении А. Её можно запустить в интерактивном режиме, где пользователь сам вводит выражения, а можно командой «python interpreter.py --test» прогнать встроенные тесты.

Состав программы: функция razbit_na_tokeny (лексер), функции sravnenie, slozhenie, umnozhenie и element (парсер, каждая отвечает за свой уровень приоритета), функция okruglenie (округление), функция vychislit (строка → число) и класс OshibkaVyrazheniya для ошибок.

Результаты вычисления корректных выражений:

```
2 + 3 * (4 - 1) / 2          = 6.5
2 + 3 * 4                    = 14
(2 + 3) * 4                  = 20
10 / 4                       = 2.5
10 - 3 - 2                   = 5
-5 + 3                       = -2
+3 + 4                       = 7
-(2 + 3)                     = -5
(-5) * 2                     = -10
3.5 + 1.25                   = 4.75
0.1 + 0.2                    = 0.3
round(3.14159, 2)            = 3.14
round(10 / 3, 3)             = 3.333
round(-2.46, 1)              = -2.5
round(2.567, 1)              = 2.6
5 >= 3                       = 1
4 >= 4                       = 1
3 >= 5                       = 0
1 + 2 >= 3                   = 1
```

Реакция на ошибочный ввод:

```
5 / 0            -> Деление на ноль
(2 + 3           -> Неправильные скобки
2 + 3)           -> Неправильные скобки
2 $ 3            -> Недопустимый символ
2 ++ 3           -> Два знака подряд
2 + -3           -> Два знака подряд
2 * -3           -> Два знака подряд
--5              -> Два знака подряд
sqrt(4)          -> Неправильный синтаксис
round(1, 2, 3)   -> Неправильный синтаксис
round(2.5, -1)   -> Неправильное число знаков в round
round(2.5, 1.5)  -> Неправильное число знаков в round
```

Пример работы в интерактивном режиме:

```
>>> 2 + 3 * (4 - 1) / 2
6.5
>>> round(3.14159, 2)
3.14
>>> 5 >= 3
1
>>> 2 ++ 3
Ошибка: Два знака подряд
>>> exit
```

Обработка результатов эксперимента.

Выполнено 31 проверок (19 верных выражений и 12 ошибочных). Во всех проверках результат совпал с ожидаемым.

Выводы.

В ходе работы написан интерпретатор арифметических выражений на Python без eval() и sympy. Он состоит из лексера и парсера рекурсивного спуска, который сразу вычисляет результат. Интерпретатор правильно учитывает приоритеты, скобки, унарный минус, целые и дробные числа, поддерживает функцию `round(x, n)` и оператор `>=`, а на некорректный ввод отвечает понятной ошибкой.

Разделение на лексер и парсер удобно: чтобы добавить оператор, достаточно новый токен в лексере и новый уровень в парсере, а чтобы добавить функцию — новую ветку в функции element.

https://github.com/vladimir45632109/laba1.git

Приложение А. Листинг программы interpreter.py

```
import sys


class OshibkaVyrazheniya(Exception):
    pass


def razbit_na_tokeny(stroka):
    tokeny = []
    i = 0
    while i < len(stroka):
        c = stroka[i]
        if c == " ":
            i += 1
        elif c in "0123456789.":
            start = i
            while i < len(stroka) and stroka[i] in "0123456789.":
                i += 1
            try:
                tokeny.append(float(stroka[start:i]))
            except ValueError:
                raise OshibkaVyrazheniya("Неправильное число")
        elif c.isalpha():
            start = i
            while i < len(stroka) and stroka[i].isalpha():
                i += 1
            tokeny.append(stroka[start:i].lower())
        elif stroka[i:i + 2] == ">=":
            tokeny.append(">=")
            i += 2
        elif c in "+-*/(),":
            tokeny.append(c)
            i += 1
        else:
            raise OshibkaVyrazheniya("Недопустимый символ")
    return tokeny


tokeny = []
poz = 0


def posmotret():
    if poz < len(tokeny):
        return tokeny[poz]
    return None


def vzyat():
    global poz
    poz += 1
    return tokeny[poz - 1]


def sravnenie():
    itog = slozhenie(True)
    while posmotret() == ">=":
        vzyat()
        pravoe = slozhenie(False)
        itog = 1 if itog >= pravoe else 0
    return itog


def slozhenie(znak_mozhno):
    itog = umnozhenie(znak_mozhno)
    while posmotret() == "+" or posmotret() == "-":
        deistvie = vzyat()
        pravoe = umnozhenie(False)
        if deistvie == "+":
            itog = itog + pravoe
        else:
            itog = itog - pravoe
    return itog
def umnozhenie(znak_mozhno):
    itog = element(znak_mozhno)
```

```
    while posmotret() == "*" or posmotret() == "/":
        deistvie = vzyat()
        pravoe = element(False)
        if deistvie == "*":
            itog = itog * pravoe
        else:
            if pravoe == 0:
                raise OshibkaVyrazheniya("Деление на ноль")
            itog = itog / pravoe
    return itog


def element(znak_mozhno):
    t = posmotret()

    if t == "+" or t == "-":
        if not znak_mozhno:
            raise OshibkaVyrazheniya("Два знака подряд")
        vzyat()
        znachenie = element(False)
        return -znachenie if t == "-" else znachenie

    if isinstance(t, float):
        vzyat()
        return t

    if t == "(":
        vzyat()
        znachenie = sravnenie()
        if posmotret() != ")":
            raise OshibkaVyrazheniya("Неправильные скобки")
        vzyat()
        return znachenie

    if t == "round":
        vzyat()
        if posmotret() != "(":
            raise OshibkaVyrazheniya("Неправильный синтаксис")
        vzyat()
        x = sravnenie()
        n = 0
        if posmotret() == ",":
            vzyat()
            n = sravnenie()
        if posmotret() != ")":
            raise OshibkaVyrazheniya("Неправильный синтаксис")
        vzyat()
        return okruglenie(x, n)

    raise OshibkaVyrazheniya("Неправильный синтаксис")


def okruglenie(x, n):
    if n != int(n) or n < 0:
        raise OshibkaVyrazheniya("Неправильное число знаков в round")
    return round(x, int(n))


def vychislit(stroka):
    global tokeny, poz
    tokeny = razbit_na_tokeny(stroka)
    poz = 0
    if len(tokeny) == 0:
        raise OshibkaVyrazheniya("Пустое выражение")
    itog = sravnenie()
    if poz < len(tokeny):
        if tokeny[poz] == ")":
            raise OshibkaVyrazheniya("Неправильные скобки")
        raise OshibkaVyrazheniya("Неправильный синтаксис")
    return itog


def format_rezultata(znachenie):
    znachenie = round(float(znachenie), 12)
    if znachenie.is_integer():
        return str(int(znachenie))
    return str(znachenie)
TESTY = [
```

```
    ("2 + 3 * (4 - 1) / 2", "6.5"),
    ("2 + 3 * 4", "14"),
    ("(2 + 3) * 4", "20"),
    ("10 / 4", "2.5"),
    ("10 - 3 - 2", "5"),
    ("-5 + 3", "-2"),
    ("+3 + 4", "7"),
    ("-(2 + 3)", "-5"),
    ("(-5) * 2", "-10"),
    ("3.5 + 1.25", "4.75"),
    ("0.1 + 0.2", "0.3"),
    ("round(3.14159, 2)", "3.14"),
    ("round(10 / 3, 3)", "3.333"),
    ("round(-2.46, 1)", "-2.5"),
    ("round(2.567, 1)", "2.6"),
    ("5 >= 3", "1"),
    ("4 >= 4", "1"),
    ("3 >= 5", "0"),
    ("1 + 2 >= 3", "1"),
]


TESTY_OSHIBOK = [
    "5 / 0",
    "(2 + 3",
    "2 + 3)",
    "2 $ 3",
    "2 ++ 3",
    "2 + -3",
    "2 * -3",
    "--5",
    "sqrt(4)",
    "round(1, 2, 3)",
    "round(2.5, -1)",
    "round(2.5, 1.5)",
]


def zapusk_testov():
    proshlo = 0
    for vyrazhenie, ozhidalos in TESTY:
        poluchilos = format_rezultata(vychislit(vyrazhenie))
        ok = (poluchilos == ozhidalos)
        proshlo += ok
        print(f"[{'OK ' if ok else 'FAIL'}] {vyrazhenie:24} = {poluchilos}")
    for vyrazhenie in TESTY_OSHIBOK:
        try:
            vychislit(vyrazhenie)
            ok, msg = False, "ошибки нет, а должна быть"
        except OshibkaVyrazheniya as e:
            ok, msg = True, str(e)
        proshlo += ok
        print(f"[{'OK ' if ok else 'FAIL'}] {vyrazhenie:24} -> {msg}")
    vsego = len(TESTY) + len(TESTY_OSHIBOK)
    print(f"\nПройдено тестов: {proshlo} из {vsego}")
    return proshlo == vsego


def dialog():
    print("Интерпретатор (вариант 10: round, >=). Для выхода введите exit")
    while True:
        try:
            vvod = input(">>> ")
        except (EOFError, KeyboardInterrupt):
            print()
            break
        if vvod.strip().lower() == "exit":
            break
        if vvod.strip() == "":
            continue
        try:
            print(format_rezultata(vychislit(vvod)))
        except OshibkaVyrazheniya as e:
            print("Ошибка:", e)


if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv[1] == "--test":
```

```
    sys.exit(0 if zapusk_testov() else 1)
dialog()
```